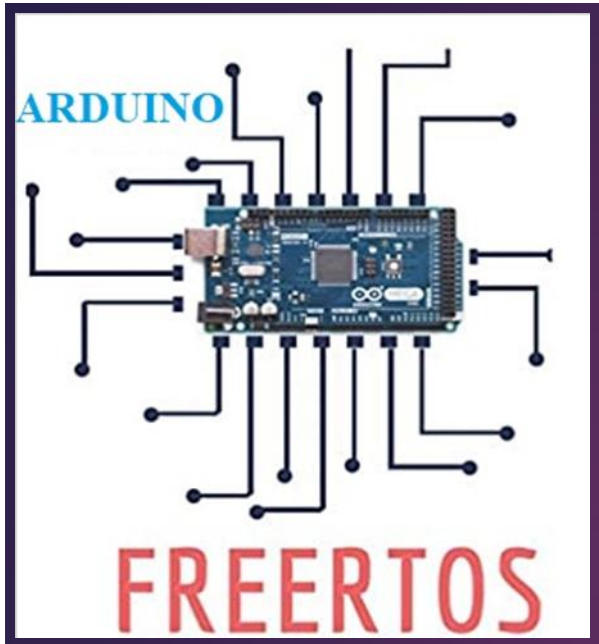


Chương 2:
Quản lý tác vụ và hàng đợi trong FreeRTOS

Nội dung

- Quản lý tác vụ FreeRTOS
- Quản lý hàng đợi FreeRTOS

Quản lý tác vụ FreeRTOS

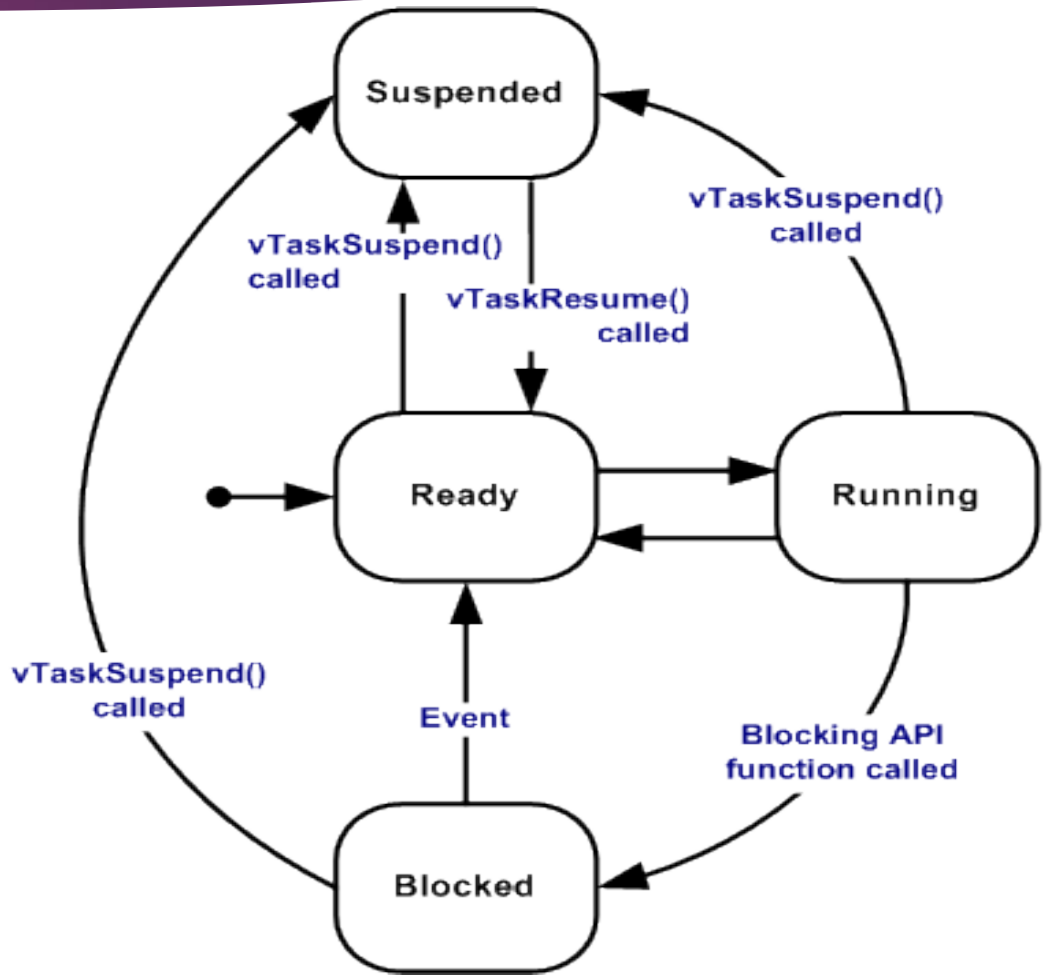
- Tác vụ trong FreeRTOS
- Các thao tác với tác vụ trong FreeRTOS
- Lập lịch tác vụ

Tác vụ trong FreeRTOS

- Một tác vụ là một chương trình, chương trình này chạy liên tục trong vòng lặp vô tận và không bao giờ dừng lại
- Trong FreeRTOS mỗi luồng thực thi được gọi là tác vụ,
- Một chương trình thường sẽ có nhiều tác vụ con khác nhau
- Ví dụ như máy bán đồ uống tự động sẽ có các thành tác vụ sau:
  + Tác vụ quản lý việc lựa chọn của người dùng
  + Tác vụ để kiểm tra đúng số tiền người dùng đã trả
  + Tác vụ để điều khiển động cơ/cơ cấu cung cấp nước uống.

Các trạng thái của một tác vụ

- **Ready:** Tác vụ đã sẵn sàng để có thể thực thi nhưng chưa được thực thi do có các tác vụ khác với độ ưu tiên ngang bằng hoặc cao hơn đang chạy.
- **Running:** khi tác vụ thực sự đang chạy
- **Blocked(Waiting):** Tác vụ đang đợi một event tạm hoặc event từ bên ngoài
- **Suspended:** Tác vụ không khả dụng để lên lịch (scheduling)



Các thao tác với tác vụ trong FreeRTOS

- *Tạo tác vụ mới*
- *Thay đổi mức ưu tiên của tác vụ*
- *Đình chỉ tác vụ*
- *Phục hồi tác vụ bị đình chỉ*
- *Xoá tác vụ*

Tạo tác vụ mới trong FreeRTOS

- Việc tạo ra các tác vụ dưới RTOS rất đơn giản.
- Một tác vụ đơn giản chỉ là một thủ tục con. Tại một số điểm trong chương trình, ta thực hiện một hoặc nhiều lời gọi tới một hàm trong RTOS để bắt đầu các tác vụ.
- Từ một task function có thể tạo ra nhiều task khác nhau.
- Mỗi task khi được tạo sẽ được cấp phát tài nguyên (bao gồm bộ nhớ stack và các biến cục bộ được khai báo trong task function) để có thể thực thi một cách độc lập. Riêng các biến static sẽ được share giữa các task
- Với FreeRTOS: Các tác vụ được khởi tạo sử dụng hàm API xTaskCreate().

Tạo tác vụ mới trong FreeRTOS

Nguyên mẫu hàm API xTaskCreate():

```
BaseType_t   xTaskCreate(TaskFunction_t  task_function,
                         const char * const  task_name,
                    uint16_t   stack_depth,
                         void  *param,
                         UBaseType_t  priority,
                         TaskHandle_t   *task_handler
                    );
```

Return:

+ pdPASS – task được tạo thành công;
+ pdFAIL – task không được tạo do thiếu bộ nhớ heap

- Trong đó:
  - ***task_function***: Con trỏ đến task function được dùng để tạo task mới;
  - ***task_name***: Con trỏ đến string chứa tên của task. Kích thước tối đa của task name được quy định trong hằng số configMAX_TASK_NAME_LEN (trong file config của FreeRTOS);
  - ***stack_depth***: Độ lớn của stack được cấp phát cho task, Idle task sử dụng stack_depth được quy định bởi hằng số configMINIMAL_STACK_SIZE và stack_depth của task tạo ra phải lớn hơn giá trị này.
  - ***param***: Con trỏ đến đối số được truyền cho task, kiểu pointer to void.
  - ***priority***: Mức độ ưu tiên của task được tạo, với 0 là mức thấp nhất (của Idle task), max là configMAX_PRIORITIES – 1.
  - ***task_handler***: Con trỏ đến task sẽ được tạo, được dùng để truyền vào các API như vTaskDelete(), vTaskPrioritySet(). Có thể truyền vào NULL nếu không cần sử dụng.

DEMO

Tạo tác vụ trước khi khởi động bộ lập lịch với Arduino Uno

```
1. #include <Arduino_FreeRTOS.h>
2. #include <task.h>
3.
4. TaskHandle_t xTaskHandle;
5. void setup() {
6.   // put your setup code here, to run once:
7.   Serial.begin(9600);
8.   xTaskCreate(Task1, "Task1", 64, NULL, 1, &xTaskHandle);
9.   delay(10);
10.  vTaskStartScheduler();
11.}
12.
13.void Task1(void * pvParameters) {
14.  for (;;) {
15.    Serial.println("Task 1 is running");
16.    delay(1000);
17.  }
18.  vTaskDelete(NULL);
19.}
```

DEMO

Tạo tác vụ sau khi khởi động bộ lập lịch từ một tác vụ khác

```
1.void Task1(void * pvParameters) {
2.  xTaskCreate(Task2, "Task2", 64, NULL, 1, &xTaskHandle2);
3.  for (;;) {
4.    Serial.println("Task 1 is running");
5.    delay(1000);
6.  }
7.  vTaskDelete(NULL);
8.}
9.
10.void Task2(void *  pvParameters) {
11. for (;;) {
12.    Serial.println("Task 2 is running");
13.    delay(1000);
14.  }
15.  vTaskDelete(NULL);
16.}
```

Thay đổi mức ưu tiên của tác vụ

- Mức ưu tiên có thể được thay đổi bởi một sự kiện (event) bên ngoài hoặc bởi một tác vụ đang chạy
- Trong FreeRTOS, sử dụng các hàm chức năng sau để thay đổi mức ưu tiên của một tác vụ:
  + Hàm API vTaskPrioritySet(): thay đổi mức ưu tiên
  + Hàm API uxTaskPriorityGet(): lấy giá trị mức ưu tiên

Thay đổi mức ưu tiên của tác vụ

- Nguyên mẫu hàm API vTaskPrioritySet():

```
void vTaskPrioritySet(TaskHandle_t   Task_handler, UBaseType_t  new_priority );
```

- Nguyên mẫu hàm API uxTaskPriorityGet():

```
void uxTaskPriorityGet(TaskHandle_t  Task_handler);
```

DEMO

Thay đổi mức ưu tiên của tác vụ

```
#include <Arduino_FreeRTOS.h>
#include <task.h>

volatile int num = 0;
TaskHandle_t xTaskHandle1;
TaskHandle_t xTaskHandle2;
BaseType_t xReturn;
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  xTaskCreate(Task1, "Task1", 64, NULL, 2, &xTaskHandle1);
  xTaskCreate(Task2, "Task2", 64, NULL, 1, &xTaskHandle2);
  delay(10);
  vTaskStartScheduler();
}

void loop() {
  // put your main code here, to run repeatedly:

}
```

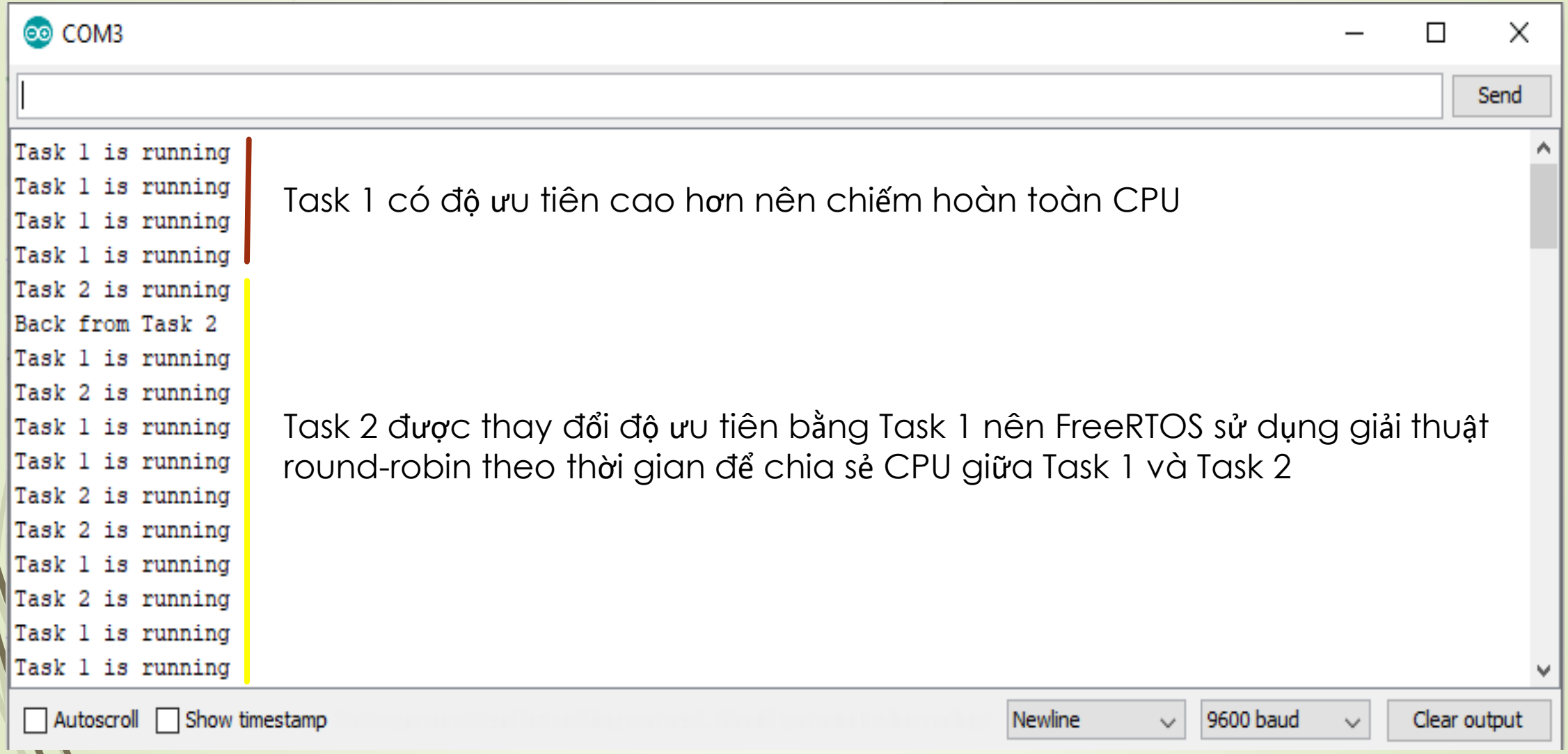
DEMO

Thay đổi mức ưu tiên của tác vụ

```
20.void Task1(void * px) {
21.  for (;;) {
22.    num++;
23.    Serial.println("Task 1 is running");
24.    delay(1000);
25.    if (num == 4) {
26.      vTaskPrioritySet(xTaskHandle2, 3);
27.      Serial.println("Back from Task 2");
28.    }
29.  }
30.  vTaskDelete(NULL);
31.}
32.void Task2(void * px) {
33.  vTaskPrioritySet(NULL, 2);
34.  for (;;) {
35.    Serial.println("Task 2 is running");
36.    delay(1000);
37.  }
38.  vTaskDelete(NULL);
39.}
```

DEMO

Thay đổi mức ưu tiên của tác vụ



Task 1 có độ ưu tiên cao hơn nên chiếm hoàn toàn CPU

Task 2 được thay đổi độ ưu tiên bằng Task 1 nên FreeRTOS sử dụng giải thuật round-robin theo thời gian để chia sẻ CPU giữa Task 1 và Task 2

Đình chỉ tác vụ

- FreeRTOS sử dụng hàm API vTaskSuspend() để đình chỉ một tác vụ.
- INCLUDE_vTaskSuspend phải được định nghĩa là 1 để chức năng này khả dụng .
- Nguyên mẫu API vTaskSuspend():

```
void  vTaskSuspend(TaskHandle_t  xTaskToSuspend);
```

Trong đó xTaskToSuspend là tham số Task_handler của hàm API xTaskCreate() khi tạo tác vụ muốn đình chỉ. Truyền là NULL nếu tác vụ gọi API muốn tự đình chỉ chính nó.

DEMO

Đình chỉ tác vụ

```
1. void vAFunction( void )
2.  {
3.  TaskHandle_t xHandle;
4.
5.      // Create a task, storing the handle.
6.      xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle );
7.
8.      // Use the handle to suspend the created task.
9.      vTaskSuspend( xHandle );
10.
11.     // The created task will not run during this period, unless
12.     // another task calls vTaskResume( xHandle ).
13.
14.     // Suspend ourselves.
15.     vTaskSuspend( NULL );
16.
17.     // We cannot get here unless another task calls vTaskResume
18.     // with our handle as the parameter.
19. }
```

Phục hồi tác vụ bị đình chỉ

- Một tác vụ đã bị đình chỉ bởi một hoặc nhiều lần gọi đến hàm API vTaskSuspend() sẽ được chạy lại bằng một lần gọi đến hàm API vTaskResume().
- INCLUDE_vTaskSuspend phải được định nghĩa là 1 để chức năng này khả dụng .
- Nguyên mẫu hàm vTaskResume():

```
void vTaskResume( TaskHandle_t  xTaskToResume );
```

Trong đó xTaskToResume là tham số Task_handler của hàm API xTaskCreate() khi tạo tác vụ muốn khôi phục.

Xoá tác vụ

- Một tác vụ có thể sử dụng hàm API vTaskDelete() để xóa chính nó hoặc bất kỳ tác vụ khác.
- Các tác vụ đã xóa không còn tồn tại và không thể nhập vào trạng thái “Running” nữa.
- Đó là trách nhiệm của tác vụ nhàn rỗi (idle) để giải phóng bộ nhớ đã cấp phát cho các tác vụ bị xóa.
- Nguyên mẫu của hàm API vTaskDelete():

```
void vTaskDelete(xTaskHandle  xTaskToDelete);
```

Lập lịch tác vụ

- Mục đích của thao tác lập lịch tác vụ là chọn ra một thứ tự tác vụ được sử dụng CPU sao cho hiệu suất sử dụng CPU là tối ưu nhất.
- FreeRTOS lựa chọn tác vụ nào được sử dụng CPU tại mỗi thời điểm (đi vào trạng thái Running) dựa trên:
  - + Mức ưu tiên của tác vụ
  - + Trạng thái hiện tại của tác vụ

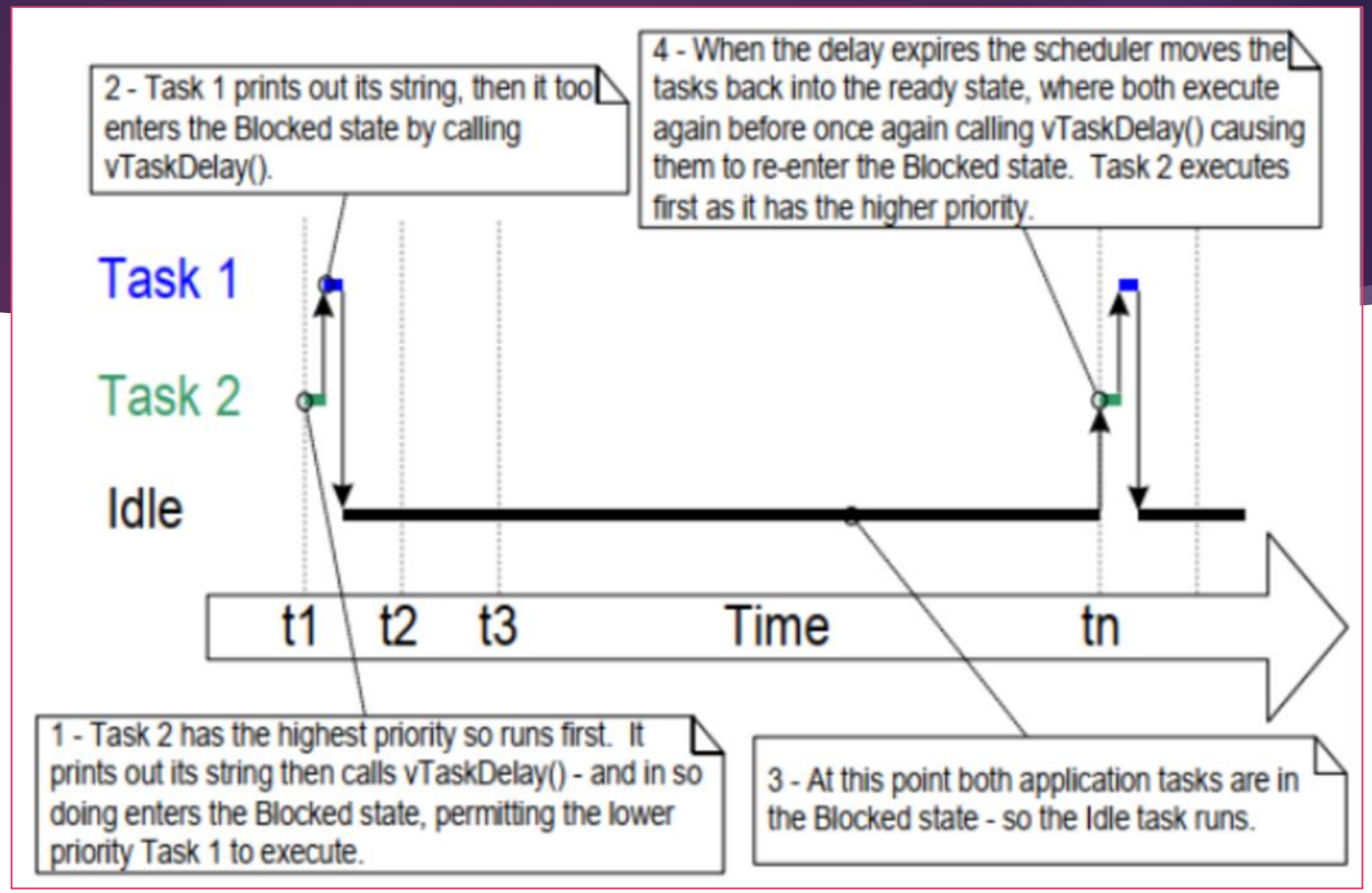
Lập lịch tác vụ

- Tại mỗi thời điểm chỉ có thể có duy nhất một tác vụ tồn tại ở trạng thái Running.
- Trình lập lịch luôn lựa chọn một tác vụ trạng thái Ready mức ưu tiên cao nhất để nhập vào trạng thái Running.
- Nếu một tác vụ đang chạy, có một tác vụ khác ưu tiên cao hơn được kích hoạt thì RTOS sẽ dừng tác vụ đang chạy và sẽ chạy tác vụ ưu tiên cao hơn kia. Tác vụ có mức ưu tiên thấp hơn sẽ bị khoá (Block)

Lập lịch tác vụ

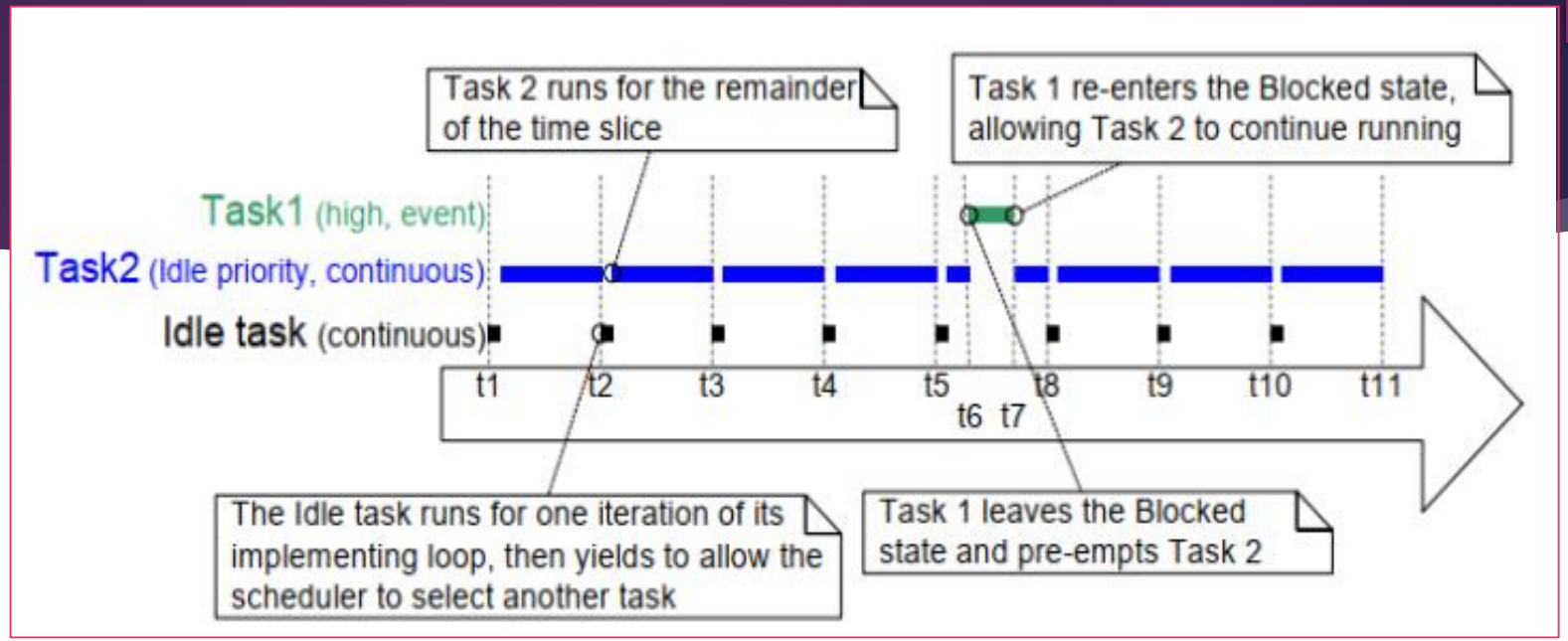
- FreeRTOS sử dụng kiểu lập lịch gọi là "*Lập lịch thay thế mức ưu tiên cố định*" (Fixed Pritority Pre-emptive Scheduling).
- "*Thay thế*" – "Pre-emptive" vì một tác vụ nhập vào trạng thái Ready hoặc đang thay đổi mức ưu tiên của nó sẽ luôn luôn thay thế tác vụ trạng thái Running nếu tác vụ trạng thái Running có mức ưu tiên thấp hơn.
- "*Mức ưu tiên cố định*" vì mỗi tác vụ được đăng kí một mức ưu tiên mà không bị thay đổi bởi chính nhân của nó.
- Nếu các tác vụ Ready đều có mức ưu tiên giống nhau, FreeRTOS sử dụng giải thuật lập lịch ***Round-Robin*** theo khe thời gian để phân chia thời gian sử dụng CPU giữa các tác vụ.

Idle Task



- Tại một thời điểm chỉ có một task được thực thi. Để đảm bảo điều này, khi API vTaskStartScheduler() được gọi, nó sẽ tạo một Idle task có mức độ ưu tiên 0 với chức năng là “không làm gì cả”.
- Vì có mức độ ưu tiên thấp nhất => Idle task không cản trở bất kỳ task nào có độ ưu tiên cao hơn vào trạng thái Running.

Idle Task



- Nếu có nhiều task có cùng độ ưu tiên với Idle task (độ ưu tiên 0), hằng số configIDLE_SHOULD_YIELD sẽ quy định cách mà Idle task sẽ thực thi:
  - Nếu configIDLE_SHOULD_YIELD = 0, Idle task sẽ thực thi nhiều lần đến khi hết time slice của nó, trừ khi bị chiếm quyền thực thi bởi task có độ ưu tiên cao hơn, sau đó lần lượt các task khác có độ ưu tiên 0 được thực thi theo time slice của riêng mình;
  - Nếu configIDLE_SHOULD_YIELD = 1, Idle task sẽ chỉ thực thi một lần rồi nhường lại cho 1 task khác có cùng độ ưu tiên 0 thực thi đến khi hết time slice đó, tiếp theo đó lần lượt các task khác có độ ưu tiên 0 được thực thi theo time slice của riêng mình (Hình vẽ)

Idle Task

- Một số lưu ý quan trọng:
  - Idle task chịu trách nhiệm giải phóng bộ nhớ mà kernel đã cấp phát cho task sau khi task bị xóa bởi API vTaskDelete(). Do đó Idle task phải được phép thực thi ngay khi có thể.
  - Chỉ những vùng nhớ được kernel cấp phát cho task mới được giải phóng tự động, những tài nguyên mà bản thân task cấp phát phải được giải phóng một cách rõ ràng, minh bạch.
- Để thêm chức năng cho Idle task, ta sử dụng Idle Task Hook Function – một callback function được Idle task gọi một lần trong mỗi vòng lặp của Idle task và thay đổi configUSE_IDLE_HOOK thành 1. Idle Task Hook Function bắt buộc phải có tên như sau:

```
void   vApplicationIdleHook(void);
```

- Có thể sử dụng Idle task để chạy các tác vụ đơn giản trên background. Tuy nhiên cần đảm bảo rằng Idle Task Hook Function không được chứa vòng lặp vô tận, vì Idle task cần được tiếp tục thực thi để làm nhiệm vụ giải phóng tài nguyên khi cần.

Bài tập thực hành (TH2) – Task Management

- TH2.1: Tạo và xóa tác vụ:
  - STM32: https://www.youtube.com/watch?v=nYlpeApGXwQ&list=PLEfMFrwVdbPYzMgeaLiFRb4ogjV8m3lt6&index=3
  - Arduino: https://microcontrollerslab.com/freertos-arduino-how-to-delete-tasks-with-vtaskdelete-api/
- TH2.2: Task suspend, resume:
  - STM32:

https://www.youtube.com/watch?v=cV_DEoA0c4Y&list=PLEfMFrwVdbPYzMgeaLiFRb4ogjV8m3lt6&index=5

  - Arduino: https://www.youtube.com/watch?v=UR4Aat6WQJY&list=PLS1QulWo1RIbpujtnhn5LRPvYYj_WqbiZ&index=3
- TH2.3: Thay đổi mức ưu tiên của tác vụ:
  - https://microcontrollerslab.com/changing-task-priority-using-freertos-arduino/

Quản lý hàng đợi FreeRTOS

- Hàng đợi trong FreeRTOS
- Các đặc điểm của hàng đợi
- Các thao tác với hàng đợi

Hàng đợi trong FreeRTOS

- Hàng đợi là nơi lưu trữ dữ liệu của các Task,
- Không gian hàng đợi có giới hạn và do người dùng định nghĩa.
- Hàng đợi hoạt động theo nguyên tắc FIFO (First In First Oute - Vào trước ra trước)

Các đặc điểm của hàng đợi

- Lưu trữ dữ liệu
- Truy cập bởi nhiều tác vụ
- Khoá quyền đọc hàng đợi
- Khoá quyền ghi hàng đợi

Lưu trữ dữ liệu

- Một hàng đợi có thể chứa một số lượng hữu hạn các phần tử được khai báo
- Số lượng tối đa phần tử mà một hàng đợi chứa được gọi là độ dài của hàng đợi
- Thông thường các hàng đợi được dùng như bộ nhớ đệm First In First Out (FIFO) nơi mà dữ liệu được ghi vào cuối hàng đợi và lấy ra ở đầu hàng đợi.

Truy cập bởi nhiều tác vụ

- Hàng đợi không được sở hữu bởi một tác vụ cụ thể nào cả mà được truy cập từ nhiều tác vụ (Task).
- Thông thường một hàng đợi được ghi từ nhiều tác vụ, và được đọc ở một số tác vụ nào đấy.
- Bất kỳ số lượng tác vụ nào cũng có thể ghi vào cùng một hàng đợi và bất kỳ số tác vụ nào có thể đọc từ cùng một hàng đợi.

Khoá quyền đọc hàng đợi

- Khi một tác vụ ra lệnh đọc một hàng đợi,nếu hàng đợi đang trống, task sẽ đi vào chế độ block và chờ. Đây là thời gian tác vụ có thể được giữ ở trạng thái Block để đợi dữ liệu khả dụng từ hàng đợi nếu hàng đợi đã bị trống.
- Một tác vụ sẽ thoát ra khỏi chế độ Block khi một tác vụ khác hoặc ISR (Interrupt Service Routine) nào đó thực hiện lệnh ghi vào hàng đợi này
- Tác vụ cũng sẽ đi đến trạng thái Ready nếu thời gian chờ kết thúc .
- Trong trường hợp nhiều tác vụ đang đọc, chỉ có một task được unblock, là tác vụ đó có ưu tiên cao nhất. Nếu chúng cùng ưu tiên, tác vụ nào chờ lâu hơn thì sẽ được ưu tiên trước.

Khoá quyền ghi hàng đợi

- Khi một tác vụ ra lệnh ghi vào hàng đợi, nó sẽ đi vào chế độ Block nếu hàng đợi đang đầy.
- Tác vụ được mở khóa (Unlocked) sẽ luôn luôn là tác vụ có mức ưu tiên cao nhất đang đợi không gian khả dụng. Nếu các tác vụ bị khóa có mức ưu tiên bằng nhau thì tác vụ đang đợi không gian lâu nhất được Unlocked => Được ghi vào hàng đợi

Các thao tác với hàng đợi

- Khởi tạo hàng đợi
- Hàm ghi dữ liệu vào hàng đợi
- Hàm đọc giữ liệu từ hàng đợi

Khởi tạo hàng đợi

- FreeRTOS sử dụng hàm để khởi tạo hàng đợi.
- Nguyên mẫu hàm xQueueCreate():

```
xQueueHandle xQueueCreate(unsigned portBASE_TYPE uxQueueLength,portBASE_TYPE uxItemSize);
```

uxQueueLength: là độ dài của hàng đợi, là số phần tử tối đa mà hàng đợi có thể chứa.
uxItemSize : là kích thước của một phần tử trong hàng đợi (kiểu của phần tử).
Giá trị trả về của hàm này:

+ NULL nếu mà khởi tạo không thành công (không đủ bộ nhớ chẳng hạn).

+ Khác NULL nếu khởi tạo thành công, lúc này giá trị bộ nhớ chính là "Handle" của hàng đợi.

Khởi tạo hàng đợi

- **Ví dụ :**

```
#include <Arduino  FreeRTOS.h>
#include <task.h>
#include <queue.h>

QueueHandle  t queue;

void setup() {
  Serial.begin(9600);
  queue = xQueueCreate( 10, sizeof( int ) );

  if(queue == NULL){
    Serial.println("Lỗi tạo hàng đợi");
  }
  Serial.println("Tạo ok");
}

void loop() {
  // put your main code here, to run repeatedly:

}
```

Hàm ghi dữ liệu vào hàng đợi

- FreeRTOS sử dụng hàm xQueueSendToFront() để ghi dữ liệu vào hàng đợi
- Nguyên mẫu của hàm xQueueSendToFront():

```
portBASE_TYPE xQueueSendToFront(xQueueHandle xQueue, const void * pvItemToQueue,portTickType xTicksToWait);
```

xQueue: Handle của hàng đợi mà mình muốn ghi vào.

pvItemToQueue: dữ liệu muốn ghi vào hàng đợi.

xTicksToWait: Số Tick trễ cho phép để time out. Nếu cái này = 0 tức là hàm này phải thực hiện ngay.

Giá trị trả về của các hàm này là:

+ pdPASS: nếu nhiệm vụ thành công.
+ errQUEUEFULL: nhiệm vụ thất bại (đầy bộ nhớ hoặc hết thời gian chờ).

Hàm đọc dữ liệu từ hàng đợi

- FreeRTOS sử dụng hàm xQueueReceive() để đọc dữ liệu từ hàng đợi
- Nguyên mẫu của hàm xQueueReceive():

portBASE_TYPE xQueueReceive(xQueueHandle xQueue, const void * pvBuffer, portTickType xTicksToWait);

- Hàm này là đọc dữ liệu mà không xóa mất dữ liệu trong hàng đợi.
  pvBuffer: Con trỏ bộ nhớ nơi mà dữ liệu được copy ra.
  Giá trị trả về của các hàm này:
  + pdPASS: Nếu nhiệm vụ thành công.
  + errQUEUE_EMPTY: nhiệm vụ thất bại (không có dữ liệu trong hàng đợi trước khi hết thời gian).

Các hàm truy vấn trong hàng đợi

- FreeRTOS sử dụng hàm uxQueueMessagesWaiting() và uxQueueMessagesWaitingFromISR() để trả về số lượng phần tử được lưu trữ trong hàng đợi
- Nguyên mẫu của hàm xQueueReceive():

```
unsigned portBASE_TYPE uxQueueMessagesWaiting ( xQueueHandle xQueue);
unsigned portBASE_TYPE uxQueueMessagesWaitingFromISR ( xQueueHandle
xQueue);
```

Giá trị trả về của các hàm này:

+ Số lượng “word” (phần tử) đang có trong hàng đợi.

Xóa/reset hàng đợi

- Sử dụng hàm xQueueDelete() để xóa hàng đợi, giải phóng tất cả bộ nhớ đã được cấp phát để lưu các phần tử của hàng đợi
- Sử dụng hàm xQueueReset() để đặt lại một hàng đợi về trạng thái trống ban đầu.
- Nguyên mẫu của hàm :
  - `void vQueueDelete (QueueHandle_t xQueue);`
  - `BaseType_t xQueueReset (QueueHandle_t xQueue);`

DEMO

Ví dụ về quản lý Queue

```
1. void loop() {
2.   if(queue == NULL)return;
3.   for(int i = 0; i<5; i++){
4.     xQueueSend(queue, &i, 0);
5.   }
6.   int element;
7.   for(int i = 0; i<5; i++){
8.     Serial.println("----------------------");
9.     xQueueReceive(queue, &element, 0);
10.    int messagesWaiting = uxQueueMessagesWaiting(queue);
11.    int emptySpaces = uxQueueSpacesAvailable(queue);
12.Serial.print("Số lượng phần tử trong hàng đợi: ");
13.    Serial.println(messagesWaiting);
14.    Serial.print("Khoảng trống: ");
15.    Serial.println(emptySpaces);
16.  }
17.  Serial.println();
18.  delay(1000);
19.}
```

Các lưu ý đặc biệt

- Nếu các hàm Sender có Priority thấp hơn hàm Receiver thì hàng đợi luôn có ít hơn 1 phần tử dữ liệu, và hàm Sender thì không cần đặt time out vì nó sẽ thực hiện ngay lập tức. Ngược lại, nếu là cao hơn.
- Với kiểu dữ liệu là kiểu cấu trúc thì phải là các trường dữ liệu để nhận diện được kiểu cấu túc đó của Sender nào gửi tới.
- Nếu dữ liệu cần lưu vào hàng đợi lớn hơn khả năng của hàng đợi thì ta chỉ cần lưu vào hàng đợi con trỏ nơi chứ dữ liệu đó.
- Tìm hiểu thêm về: xQueueOverwrite(), xQueuePeek()
- Phân biệt: xQueueSend(), xQueueSendToFront(), xQueueSendToBack()

Bài tập thực hành (TH3)- Queue

- TH3.1:

https://www.youtube.com/watch?v=elgkseFUpmk&list=PLEfMFrwVdbPYzMgeaLiFRb4ogjV8m3lt6&index=9

- TH3.2:

https://www.youtube.com/watch?v=Z-XD3Q7Hqps&list=PLEfMFrwVdbPYzMgeaLiFRb4ogjV8m3lt6&index=10

Bài tập tổng hợp (TH4) – Task & Queue

- TH4.1: Bật tắt Led

"*FreeRTOS with Arduino Tutorial 2 – How to Handle Tasks with FreeRTOS on Arduino*" – Youtube

https://www.youtube.com/watch?v=KjsMTxJVT2M&t=712s

Bài tập tổng hợp (TH4) – Task & Queue

- TH4.2:

Viết chương trình thực hiện tạo hai tác vụ có tên “Task1”, “Task2” sao cho:

- Cả 2 tác vụ có: Stack 250 từ máy (word), mức ưu tiên (tskIDLE_PRIORITY + 1), Handle lần lượt là HTask1, HTask2
- Task1 thực hiện tạo hàng đợi có tên “Queue” có 6 phần tử kiểu chuỗi 20 ký tự; gửi lần lượt 3 chuỗi ký tự (“CT03-Hello”; “DT02-Hi”; “AT-Welcome” cho trước) vào đầu hàng đợi.
- Task2 đọc từng phần tử từ hàng đợi và lần lượt in ra màn hình sau mỗi lần đọc: giá trị của phần tử; số phần tử có thể đọc ra từ hàng đợi và không gian khả dụng còn lại tương ứng của hàng đợi.

Bài tập tổng hợp (TH4) – Task & Queue

- TH4.3:

Viết chương trình tạo 2 tác vụ có mức ưu tiên bằng nhau:

- Task1: In ra “Hello, world!”
- Task2: In ra “Bye Task1” và đình chỉ task 1 trong 10s
- Task1 thực hiện 5 lần -> Task2 -> Lặp lại
- In ra mức ưu tiên của các task

Q & A